МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ №7

з дисципліни «Професійний Пайтон»

на тему: «Робота з базами даних у Python»

Виконав:
студент групи ФеП-32
Накладюк Володимир Тарасович
Варіант №15

Львів - 2026

1. Мета роботи

Ознайомитися з організацією шару зберігання даних у Python-застосунках: реляційними базами даних, мовою SQL, СУБД SQLite, стандартом DB-API, параметризованими запитами, CRUD, транзакціями та принципами ACID, патерном Repository, SQLAlchemy ORM і міграціями Alembic. Інтегрувати persistence layer у проєкт попередніх робіт і перевірити його integration tests на окремій тестовій базі.

2. Індивідуальне завдання

Варіант 15. Облік результатів наукових експериментів.

- продовжити проєкт Lab_01: для experiment зберігати назву, researcher, параметр, результат, дату;
- створити таблиці researchers і experiments; реалізувати ResearcherRepository та ExperimentRepository;
- зберегти функції Lab_01 (додавання, пошук за researcher, середнє/min/max, фільтрація за діапазоном), додати CRUD і сортування за датою;
- через DB-API реалізувати параметризований пошук за researcher або діапазоном результатів; використати AVG(), MIN(), MAX(), ORDER BY;
- транзакція: пакетне додавання серії експериментів із rollback при некоректному результаті;
- міграція: додати notes; integration tests: CRUD, researcher relationship, aggregates, range filter, date ordering та rollback.

3. Постановка задачі та опис предметної області

Дослідник може провести багато експериментів, а кожен експеримент належить одному досліднику — це зв’язок «один до багатьох». Дані потрібно зберігати в SQLite, а доступ до них організувати через окремий persistence layer: ORM-моделі, репозиторії, сервіс із транзакціями та модуль прямих SQL-запитів через DB-API. Схема бази створюється й змінюється лише міграціями Alembic.

4. Хід виконання

4.1. Структура проєкту

Лістинг 7.1. Структура проєкту lab07 (основні нові частини)

```
lab07/
├── migrations/
│   ├── versions/
│   │   ├── 0001_create_researchers_and_experiments.py
│   │   └── 0002_add_experiment_notes.py
│   ├── env.py
│   └── script.py.mako
├── src/
│   └── experiment_registry/
│       ├── persistence/
│       │   ├── __init__.py
│       │   ├── database.py
│       │   ├── dbapi.py
│       │   ├── orm.py
│       │   ├── repositories.py
```

```
│       │   └── services.py
│       ├── __init__.py
│       ├── analytics.py
│       ├── config.py
│       ├── data.py
│       ├── decorators.py
│       ├── exceptions.py
│       ├── main.py
│       ├── models.py
│       ├── processors.py
│       └── services.py
├── tests/
│   ├── integration/
│   │   ├── test_application_flow.py
│   │   ├── test_dbapi_and_migrations.py
│   │   ├── test_experiment_pipeline.py
│   │   ├── test_import_pipeline.py
│   │   └── test_persistence.py
│   └── conftest.py
├── .gitignore
├── alembic.ini
├── config.yaml
├── pyproject.toml
└── README.md
```

4.2. Схема бази даних, таблиці та поля

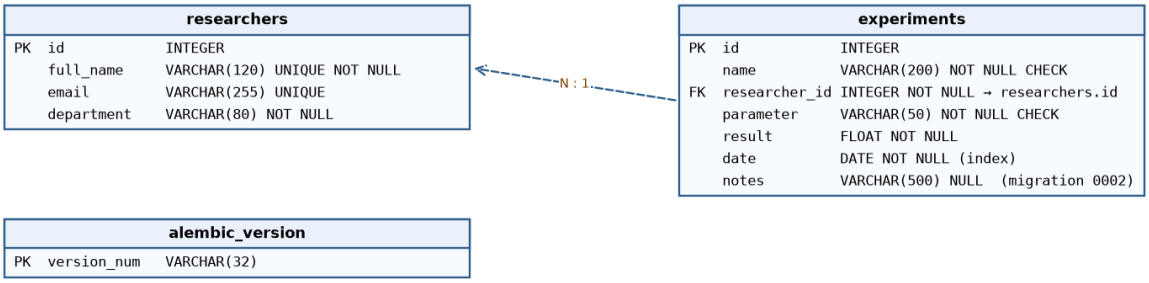


Рис. 7.1. Схема бази даних (ER-діаграма): experiments.researcher_id → researchers.id, зв’язок N : 1, ON DELETE CASCADE

Таблиця 7.1. Перелік таблиць і опис полів

Таблиця	Поле	Тип	Обмеження	Опис
researchers	id	INTEGER	PRIMARY KEY	ідентифікатор дослідника
	full_name	VARCHAR(120)	NOT NULL, UNIQUE	ім’я та прізвище
	email	VARCHAR(255)	UNIQUE, NULL	електронна пошта
	department	VARCHAR(80)	NOT NULL	підрозділ
experiments	id	INTEGER	PRIMARY KEY	ідентифікатор експерименту
	name	VARCHAR(200)	NOT NULL, CHECK length > 0	назва експерименту
	researcher_id	INTEGER	FOREIGN KEY → researchers.id, ON DELETE CASCADE, index	дослідник
	parameter	VARCHAR(50)	NOT NULL, CHECK length > 0	вимірюваний параметр
	result	FLOAT	NOT NULL	числовий результат
	date	DATE	NOT NULL, index	дата експерименту

Таблиця	Поле	Тип	Обмеження	Опис
	notes	VARCHAR(500)	NULL	примітка (міграція 0002)

Primary Keys. У кожній таблиці штучний ключ `id INTEGER PRIMARY KEY`; у SQLite він є псевдонімом `rowid`, тому значення генерується автоматично. **Foreign Key.** `experiments.researcher_id` посилається на `researchers.id`: не можна створити експеримент неіснуючого дослідника, а видалення дослідника каскадно видаляє його експерименти. SQLite перевіряє зовнішні ключі лише після `PRAGMA foreign_keys = ON`, тому прагма вмикається для кожного нового з’єднання.

4.3. SQL-команди створення таблиць

Лістинг 7.2. Еквівалентний SQL (SCHEMA_SQL у dbapi.py)

```
CREATE TABLE researchers (
    id          INTEGER PRIMARY KEY,
    full_name   VARCHAR(120) NOT NULL UNIQUE,
    email       VARCHAR(255) UNIQUE,
    department  VARCHAR(80)  NOT NULL
);

CREATE TABLE experiments (
    id            INTEGER PRIMARY KEY,
    name          VARCHAR(200) NOT NULL CHECK (length(name) > 0),
    researcher_id INTEGER      NOT NULL REFERENCES researchers (id) ON DELETE CASCADE,
    parameter     VARCHAR(50)  NOT NULL CHECK (length(parameter) > 0),
    result        FLOAT        NOT NULL,
    date          DATE         NOT NULL,
    notes         VARCHAR(500)
);
```

```
q3efirok@fedora: ~/Documents/Унік/ПП
(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab07$ python -m sqlite3 experiments.db "SELECT sql FROM
sqlite_master WHERE type = 'table' AND name IN ('researchers', 'experiments')"
('CREATE TABLE researchers (\n\tid INTEGER NOT NULL, \n\tfull_name VARCHAR(120) NOT NULL, \n\temail
VARCHAR(255), \n\tdepartment VARCHAR(80) NOT NULL, \n\tPRIMARY KEY (id), \n\tUNIQUE (email), \n\tUNIQUE
(full_name)\n)',)
('CREATE TABLE "experiments" (\n\tid INTEGER NOT NULL, \n\tname VARCHAR(200) NOT NULL, \n\tresearcher_id
INTEGER NOT NULL, \n\tparameter VARCHAR(50) NOT NULL, \n\tresult FLOAT NOT NULL, \n\tdate DATE NOT NULL, notes
 VARCHAR(500), \n\tPRIMARY KEY (id), \n\tCONSTRAINT ck_experiments_name_not_empty CHECK (length(name) > 0),
\n\tCONSTRAINT ck_experiments_parameter_not_empty CHECK (length(parameter) > 0), \n\tFOREIGN
KEY(researcher_id) REFERENCES researchers (id) ON DELETE CASCADE\n)',)
```

Рис. 7.2. Фактична схема, створена міграціями (sqlite_master)

4.4. Код database.py

Лістинг 7.3. Модуль database.py

```
"""Engine, session factory and the declarative base."""

import os
import sqlite3
from collections.abc import Iterator
from contextlib import contextmanager
from typing import Any

from sqlalchemy import Engine, create_engine, event
from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker

DEFAULT_DATABASE_URL = "sqlite:///experiments.db"


class Base(DeclarativeBase):
    """Base class of all ORM models."""
```

```
def get_database_url() -> str:
    return os.environ.get("DATABASE_URL", DEFAULT_DATABASE_URL)


def _enable_sqlite_foreign_keys(dbapi_connection: Any, _record: Any) -> None:
    """SQLite ignores FOREIGN KEY constraints unless the pragma is enabled per connection."""
    if isinstance(dbapi_connection, sqlite3.Connection):
        cursor = dbapi_connection.cursor()
        cursor.execute("PRAGMA foreign_keys = ON")
        cursor.close()


def create_db_engine(url: str | None = None, echo: bool = False) -> Engine:
    engine = create_engine(url or get_database_url(), echo=echo)
    event.listen(engine, "connect", _enable_sqlite_foreign_keys)
    return engine


def create_session_factory(engine: Engine) -> sessionmaker[Session]:
    return sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)


def init_db(engine: Engine) -> None:
    """Create all tables (used by tests; production schema is managed by Alembic)."""
    from experiment_registry.persistence import orm  # noqa: F401  (register models)

    Base.metadata.create_all(engine)


@contextmanager
def session_scope(factory: sessionmaker[Session]) -> Iterator[Session]:
    """Unit of work: commit on success, rollback on any exception."""
    session = factory()
    try:
        yield session
        session.commit()
    except Exception:
        session.rollback()
        raise
    finally:
        session.close()
```

4.5. ORM-моделі

Моделі оголошено в стилі SQLAlchemy 2.0 з `Mapped[...]` і `mapped_column`, тому типи колонок перевіряє mypy. `relationship(back_populates=...)` зв’язує обидві сторони: `researcher.experiments` повертає список експериментів (відсортований за датою), а `experiment.researcher` — власника. `cascade="all, delete-orphan"` узгоджує поведінку ORM з `ON DELETE CASCADE` у базі.

Лістинг 7.4. Модуль orm.py

```
"""ORM models: tables `researchers` and `experiments` (one-to-many)."""

from datetime import date

from sqlalchemy import CheckConstraint, Date, Float, ForeignKey, Integer, String
from sqlalchemy.orm import Mapped, mapped_column, relationship

from experiment_registry.persistence.database import Base


class ResearcherORM(Base):
    __tablename__ = "researchers"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    full_name: Mapped[str] = mapped_column(String(120), unique=True, nullable=False)
    email: Mapped[str | None] = mapped_column(String(255), unique=True, nullable=True)
    department: Mapped[str] = mapped_column(String(80), nullable=False, default="Physics")
```

```
    experiments: Mapped[list["ExperimentORM"]] = relationship(
        back_populates="researcher", cascade="all, delete-orphan",
order_by="ExperimentORM.date",
    )

    def __repr__(self) -> str:
        return f"ResearcherORM(id={self.id}, full_name={self.full_name!r})"


class ExperimentORM(Base):
    __tablename__ = "experiments"
    __table_args__ = (
        CheckConstraint("length(name) > 0", name="ck_experiments_name_not_empty"),
        CheckConstraint("length(parameter) > 0", name="ck_experiments_parameter_not_empty"),
    )

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String(200), nullable=False)
    researcher_id: Mapped[int] = mapped_column(
        ForeignKey("researchers.id", ondelete="CASCADE"), nullable=False, index=True,
    )
    parameter: Mapped[str] = mapped_column(String(50), nullable=False)
    result: Mapped[float] = mapped_column(Float, nullable=False)
    date: Mapped[date] = mapped_column(Date, nullable=False, index=True)
    notes: Mapped[str | None] = mapped_column(String(500), nullable=True)  # migration 0002

    researcher: Mapped[ResearcherORM] = relationship(back_populates="experiments")

    def __repr__(self) -> str:
        return f"ExperimentORM(id={self.id}, name={self.name!r}, result={self.result})"
```

```
q3efirok@fedora: ~/Documents/Унік/ПП
(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab07$ python -c
"print(select(ExperimentORM).join(...).where(...).order_by(...))"
SELECT experiments.id, experiments.name, experiments.researcher_id, experiments.parameter, experiments.result,
 experiments.date, experiments.notes
FROM experiments JOIN researchers ON researchers.id = experiments.researcher_id
WHERE lower(researchers.full_name) = :lower_1 ORDER BY experiments.date
```

Рис. 7.3. SQL, який SQLAlchemy генерує для ORM-запиту пошуку за дослідником

4.6. Repository Pattern

Репозиторії приховують деталі SQLAlchemy від сервісного шару: сервіс викликає `find_by_researcher()` чи `summary()`, не знаючи про `select()` і `join()`. Репозиторій працює з переданою сесією і викликає лише `flush()`, а рішення про `commit` чи `rollback` приймає unit of work (`session_scope`).

Лістинг 7.5. ExperimentRepository: CRUD, пошук, агрегати

```
class ExperimentRepository:
    def __init__(self, session: Session) -> None:
        self.session = session

    # ------------------------------------------------------------------ CRUD
    def add(self, name: str, researcher: ResearcherORM, parameter: str, result: float,
            day: date, notes: str | None = None) -> ExperimentORM:
        experiment = ExperimentORM(name=name, researcher=researcher, parameter=parameter,
                                   result=result, date=day, notes=notes)
        self.session.add(experiment)
        self.session.flush()
        return experiment

    def get(self, experiment_id: int) -> ExperimentORM:
        experiment = self.session.get(ExperimentORM, experiment_id)
        if experiment is None:
            raise ExperimentNotFoundError(f"Experiment #{experiment_id} not found.")
        return experiment

    def list_all(self, limit: int | None = None, offset: int = 0) -> Sequence[ExperimentORM]:
        """All experiments sorted by date (with optional pagination)."""
        statement = (select(ExperimentORM).options(joinedload(ExperimentORM.researcher))
                     .order_by(ExperimentORM.date, ExperimentORM.id).offset(offset))
```

```
        if limit is not None:
            statement = statement.limit(limit)
        return self.session.scalars(statement).all()

    def update(self, experiment_id: int, **fields: object) -> ExperimentORM:
        allowed = {"name", "parameter", "result", "date", "notes"}
        unknown = set(fields) - allowed
        if unknown:
            raise ValueError(f"Cannot update fields: {sorted(unknown)}")
        experiment = self.get(experiment_id)
        for key, value in fields.items():
            setattr(experiment, key, value)
        self.session.flush()
        return experiment

    def delete(self, experiment_id: int) -> None:
        self.session.delete(self.get(experiment_id))
        self.session.flush()

    # ------------------------------------------------------------ queries
    def find_by_researcher(self, full_name: str) -> Sequence[ExperimentORM]:
        statement = (
            select(ExperimentORM)
            .join(ExperimentORM.researcher)
            .options(joinedload(ExperimentORM.researcher))
            .where(func.lower(ResearcherORM.full_name) == full_name.strip().lower())
            .order_by(ExperimentORM.date)
        )
        return self.session.scalars(statement).all()

    def filter_by_result_range(self, low: float, high: float,
                               parameter: str | None = None) -> Sequence[ExperimentORM]:
        if low > high:
            raise ValueError(f"Invalid range: {low} > {high}")
        statement = (select(ExperimentORM).options(joinedload(ExperimentORM.researcher))
                     .where(ExperimentORM.result.between(low, high)))
        if parameter is not None:
            statement = statement.where(ExperimentORM.parameter == parameter)
        return self.session.scalars(statement.order_by(ExperimentORM.result)).all()

    def summary(self, parameter: str | None = None, researcher_id: int | None = None) -> ResultSummary:
        """COUNT / AVG / MIN / MAX calculated by the database."""
        statement = select(func.count(ExperimentORM.id), func.avg(ExperimentORM.result),
                           func.min(ExperimentORM.result), func.max(ExperimentORM.result))
        if parameter is not None:
            statement = statement.where(ExperimentORM.parameter == parameter)
        if researcher_id is not None:
            statement = statement.where(ExperimentORM.researcher_id == researcher_id)
        count, average, minimum, maximum = self.session.execute(statement).one()
        return ResultSummary(count, average, minimum, maximum)

    def statistics_by_researcher(self) -> list[tuple[str, int, float, float, float]]:
        """JOIN + GROUP BY: per-researcher count, average, minimum and maximum."""
        statement = (
            select(ResearcherORM.full_name, func.count(ExperimentORM.id),
func.avg(ExperimentORM.result),
                   func.min(ExperimentORM.result), func.max(ExperimentORM.result))
            .join(ExperimentORM, ExperimentORM.researcher_id == ResearcherORM.id)
            .group_by(ResearcherORM.full_name)
            .order_by(ResearcherORM.full_name)
        )
        return [(name, count, avg, low, high)
                for name, count, avg, low, high in self.session.execute(statement).all()]
```

4.7. Приклади CRUD

Таблиця 7.2. CRUD-операції репозиторію

Операція	Метод	SQL
Create	`ExperimentRepository.add()`	`INSERT INTO experiments (...) VALUES (...)`
Read	`get()`, `list_all()`, `find_by_researcher()`	`SELECT ... WHERE id = ?`, `JOIN`, `ORDER BY date`
Update	`update(id, result=..., notes=...)`	`UPDATE experiments SET ... WHERE id = ?`
Delete	`delete(id)`	`DELETE FROM experiments WHERE id = ?`

4.8. Parameterized queries через DB-API

Модуль `dbapi.py` використовує стандартний `sqlite3`. Значення передаються окремо від тексту запиту через плейсхолдери `?` або `:name`, тому драйвер ніколи не вставляє їх у SQL як код. Спроба ін’єкції `x' OR '1'='1` повертає 0 рядків, бо шукається дослідник саме з таким ім’ям.

Лістинг 7.6. Параметризовані запити з AVG, MIN, MAX, ORDER BY

```
def search_by_researcher(connection: sqlite3.Connection, researcher: str) -> list[sqlite3.Row]:
    """Parameterized query: the value is passed separately, never formatted into SQL."""
    sql = """
        SELECT e.id, e.name, r.full_name AS researcher, e.parameter, e.result, e.date
        FROM experiments AS e
        JOIN researchers AS r ON r.id = e.researcher_id
        WHERE lower(r.full_name) = lower(?)
        ORDER BY e.date
    """
    return connection.execute(sql, (researcher,)).fetchall()


def search_by_result_range(connection: sqlite3.Connection, low: float, high: float,
                           parameter: str | None = None) -> list[sqlite3.Row]:
    sql = """
        SELECT e.id, e.name, e.parameter, e.result, e.date
        FROM experiments AS e
        WHERE e.result BETWEEN :low AND :high
          AND (:parameter IS NULL OR e.parameter = :parameter)
        ORDER BY e.result
    """
    return connection.execute(sql, {"low": low, "high": high, "parameter":
parameter}).fetchall()


def aggregate_by_parameter(connection: sqlite3.Connection) -> list[sqlite3.Row]:
    sql = """
        SELECT parameter, COUNT(*) AS count, ROUND(AVG(result), 3) AS average,
               MIN(result) AS minimum, MAX(result) AS maximum
        FROM experiments
        GROUP BY parameter
        ORDER BY parameter
    """
    return connection.execute(sql).fetchall()
```

4.9. Транзакції, commit і rollback

`session_scope` реалізує unit of work: усі зміни в блоці виконуються в одній транзакції; при успішному завершенні викликається `commit()`, який робить зміни постійними, а при будь-якому винятку — `rollback()`, що скасовує всі зміни з початку транзакції. `add_series` додає серію експериментів (і нових дослідників) в одній транзакції: якщо третій запис має некоректний результат, перші два, уже записані через `flush()`, також скасовуються. Порушення обмежень бази (`IntegrityError`) перетворюється на `ExperimentValidationError` зі збереженням причини.

Лістинг 7.7. Unit of work

```
@contextmanager
def session_scope(factory: sessionmaker[Session]) -> Iterator[Session]:
    """Unit of work: commit on success, rollback on any exception."""
    session = factory()
    try:
        yield session
        session.commit()
    except Exception:
```

```
        session.rollback()
        raise
    finally:
        session.close()
```

Лістинг 7.8. Сервіс із транзакційними операціями

```
class ExperimentRegistry:
    def __init__(self, session_factory: sessionmaker[Session],
                 min_result: float = 0.0, max_result: float = 1_000_000.0) -> None:
        self._sessions = session_factory
        self._min = min_result
        self._max = max_result

    def _validate(self, item: NewExperiment) -> None:
        if not item.name.strip() or not item.researcher.strip() or not item.parameter.strip():
            raise ExperimentValidationError(f"Empty field in {item!r}")
        if not math.isfinite(item.result) or not self._min <= item.result <= self._max:
            raise ExperimentValidationError(
                f"Result {item.result} of {item.name!r} is outside [{self._min}, {self._max}]")

    def _insert(self, session: Session, item: NewExperiment) -> ExperimentORM:
        self._validate(item)
        researcher = ResearcherRepository(session).get_or_create(item.researcher)
        return ExperimentRepository(session).add(item.name, researcher, item.parameter, item.result,
                                                 item.date, item.notes)

    def add_experiment(self, item: NewExperiment) -> int:
        with session_scope(self._sessions) as session:
            return self._insert(session, item).id

    def add_series(self, items: Sequence[NewExperiment]) -> list[int]:
        """Batch insert in ONE transaction: an invalid record rolls back the whole series."""
        try:
            with session_scope(self._sessions) as session:
                return [self._insert(session, item).id for item in items]
        except IntegrityError as error:
            raise ExperimentValidationError(f"Database constraint violated: {error.orig}") from error

    def update_result(self, experiment_id: int, result: float) -> ExperimentORM:
        with session_scope(self._sessions) as session:
            experiment = ExperimentRepository(session).get(experiment_id)
            self._validate(NewExperiment(experiment.name, "-", experiment.parameter, result,
experiment.date))
            return ExperimentRepository(session).update(experiment_id, result=result)

    def delete(self, experiment_id: int) -> None:
        with session_scope(self._sessions) as session:
            ExperimentRepository(session).delete(experiment_id)

    def find_by_researcher(self, full_name: str) -> Sequence[ExperimentORM]:
        with session_scope(self._sessions) as session:
            return ExperimentRepository(session).find_by_researcher(full_name)

    def filter_by_range(self, low: float, high: float, parameter: str | None = None) ->
Sequence[ExperimentORM]:
        with session_scope(self._sessions) as session:
            return ExperimentRepository(session).filter_by_result_range(low, high, parameter)

    def sorted_by_date(self) -> Sequence[ExperimentORM]:
        with session_scope(self._sessions) as session:
            return ExperimentRepository(session).list_all()

    def statistics(self, parameter: str | None = None) -> ResultSummary:
        with session_scope(self._sessions) as session:
            return ExperimentRepository(session).summary(parameter=parameter)

    def statistics_by_researcher(self) -> list[tuple[str, int, float, float, float]]:
        with session_scope(self._sessions) as session:
            return ExperimentRepository(session).statistics_by_researcher()
```

4.10. Властивості ACID

Таблиця 7.3. ACID у persistence layer

Властивість	Як забезпечується в проєкті
Atomicity	`add_series` — одна транзакція: або зберігається вся серія, або нічого (rollback).

Властивість	Як забезпечується в проєкті
Consistency	PRIMARY/FOREIGN KEY, UNIQUE, CHECK, NOT NULL у схемі та перевірка результату в сервісі.
Isolation	Кожен виклик сервісу має власну сесію; незакомічені зміни не видно іншим з’єднанням (SQLite блокує запис на час транзакції).
Durability	Після `commit()` SQLite записує зміни на диск через журнал транзакцій; вони зберігаються після завершення програми.

4.11. Міграції Alembic

Alembic ініціалізовано командою `alembic init migrations`; у `env.py` підключено `Base.metadata`, URL з `DATABASE_URL` і batch-режим для SQLite. Міграцію 0001 (створення таблиць) і 0002 (додавання `notes`) згенеровано `alembic revision --autogenerate` за змінами моделей. Під час тестування виявлено, що стандартний `fileConfig()` у `env.py` вимикав уже створені логери застосунку, і тест журналу імпорту падав, якщо запускався після тесту міграцій; виправлено параметром `disable_existing_loggers=False`.

Лістинг 7.9. Міграція 0002: додавання notes

```
"""add experiment notes

Revision ID: 0002
Revises: 0001
Create Date: 2026-11-09 00:08:07.954369

"""
from typing import Sequence, Union

from alembic import op
import sqlalchemy as sa


# revision identifiers, used by Alembic.
revision: str = '0002'
down_revision: Union[str, Sequence[str], None] = '0001'
branch_labels: Union[str, Sequence[str], None] = None
depends_on: Union[str, Sequence[str], None] = None


def upgrade() -> None:
    """Upgrade schema."""
    # ### commands auto generated by Alembic - please adjust! ###
    with op.batch_alter_table('experiments', schema=None) as batch_op:
        batch_op.add_column(sa.Column('notes', sa.String(length=500), nullable=True))

    # ### end Alembic commands ###


def downgrade() -> None:
    """Downgrade schema."""
    # ### commands auto generated by Alembic - please adjust! ###
    with op.batch_alter_table('experiments', schema=None) as batch_op:
        batch_op.drop_column('notes')

    # ### end Alembic commands ###
```

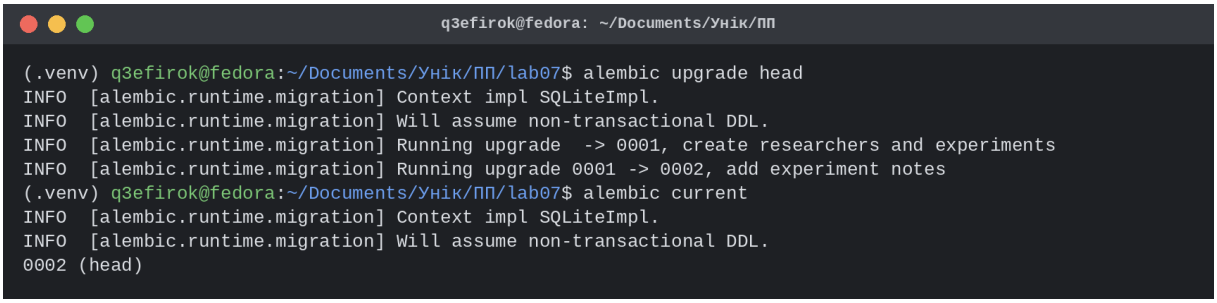


Рис. 7.4. Застосування міграцій і поточна версія

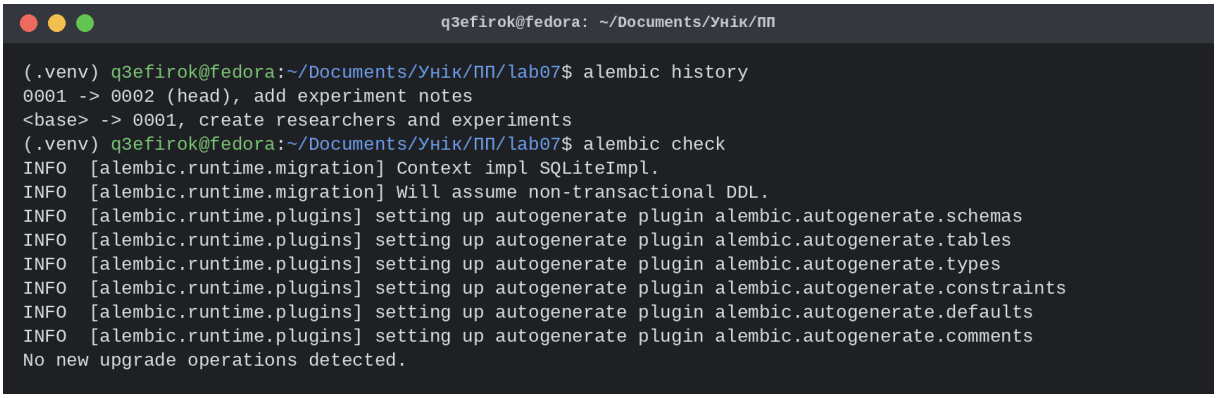


Рис. 7.5. Історія міграцій; моделі й схема синхронізовані

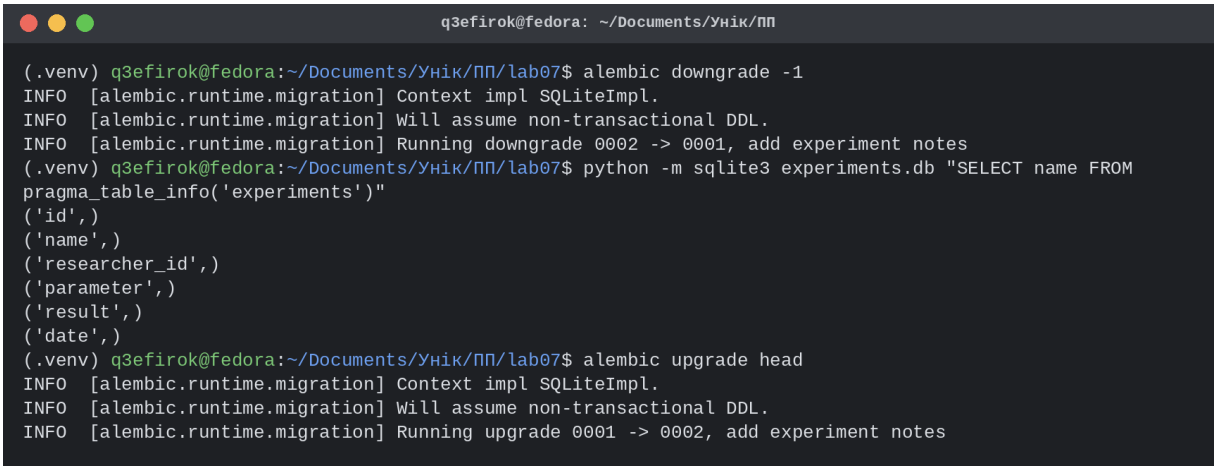


Рис. 7.6. downgrade прибирає колонку notes, upgrade повертає її

4.12. Робота програми

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab07$ experiment-registry db
CREATE: inserted 8 experiments in one transaction, IDs 1..8

READ: experiments of Olena Kovalenko
  #1   2026-02-03  Thermal conductivity of Al alloy   Olena Kovalenko  temperature    293.40
  #3   2026-02-10  Thermal conductivity of Cu         Olena Kovalenko  temperature    301.80
  #7   2026-03-02  Heat exchanger efficiency          Olena Kovalenko  temperature    348.90  [repeat with
insulation]

Aggregates (temperature): COUNT=4 AVG=255.32 MIN=77.2 MAX=348.9

Range filter: temperature in [290, 320]
  #1   2026-02-03  Thermal conductivity of Al alloy   Olena Kovalenko  temperature    293.40
  #3   2026-02-10  Thermal conductivity of Cu         Olena Kovalenko  temperature    301.80

UPDATE: experiment #2 result -> 102.0
DELETE: experiment #6 removed

Sorted by date (ORDER BY date)
  #1   2026-02-03  Thermal conductivity of Al alloy   Olena Kovalenko  temperature    293.40
  #2   2026-02-05  Gas compression test               Taras Hnatiuk    pressure       102.00
  #3   2026-02-10  Thermal conductivity of Cu         Olena Kovalenko  temperature    301.80
  #4   2026-02-12  Battery discharge cycle            Iryna Melnyk     voltage          3.71
  #5   2026-02-18  Cryogenic cooling run              Andrii Bondar    temperature     77.20
  #7   2026-03-02  Heat exchanger efficiency          Olena Kovalenko  temperature    348.90  [repeat with
insulation]
  #8   2026-03-04  Solar cell output                  Iryna Melnyk     voltage          0.62

Statistics by researcher (JOIN + GROUP BY):
  Andrii Bondar    n=1  avg=   77.20  min=   77.20  max=   77.20
  Iryna Melnyk     n=2  avg=    2.17  min=    0.62  max=    3.71
  Olena Kovalenko  n=3  avg=  314.70  min=  293.40  max=  348.90
  Taras Hnatiuk    n=1  avg=  102.00  min=  102.00  max=  102.00

TRANSACTION: Result -5.0 of 'Broken sensor run' is outside [0.0, 1000000.0]
  rows before = 7, after = 7 -> ROLLBACK, the first two records were not saved

DB-API (sqlite3), parameterized queries:
  researcher=? -> #2 Gas compression test 102.0
  BETWEEN :low AND :high -> #8 Solar cell output 0.62
  BETWEEN :low AND :high -> #4 Battery discharge cycle 3.71
  injection attempt "x' OR '1'='1": 0 rows
  pressure     COUNT=1 AVG=102.0 MIN=102.0 MAX=102.0
  temperature  COUNT=4 AVG=255.325 MIN=77.2 MAX=348.9
  voltage      COUNT=2 AVG=2.165 MIN=0.62 MAX=3.71
```

Рис. 7.7. Підкоманда db: CRUD, агрегати, транзакція з rollback і DB-API

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab07$ python -m sqlite3 experiments.db "SELECT r.full_name,
COUNT(*), ROUND(AVG(e.result), 2), MIN(e.result), MAX(e.result) FROM experiments e JOIN researchers r ON
r.id = e.researcher_id GROUP BY r.full_name ORDER BY r.full_name"
('Andrii Bondar', 1, 77.2, 77.2, 77.2)
('Iryna Melnyk', 2, 2.17, 0.62, 3.71)
('Olena Kovalenko', 3, 314.7, 293.4, 348.9)
('Taras Hnatiuk', 1, 102.0, 102.0, 102.0)
(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab07$ python -m sqlite3 experiments.db "SELECT id, date, name,
result, notes FROM experiments WHERE parameter = 'temperature' ORDER BY date"
(1, '2026-02-03', 'Thermal conductivity of Al alloy', 293.4, None)
(3, '2026-02-10', 'Thermal conductivity of Cu', 301.8, None)
(5, '2026-02-18', 'Cryogenic cooling run', 77.2, None)
(7, '2026-03-02', 'Heat exchanger efficiency', 348.9, 'repeat with insulation')
(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab07$ python -m sqlite3 experiments.db "PRAGMA
foreign_key_list(experiments)"
(0, 0, 'researchers', 'researcher_id', 'id', 'NO ACTION', 'CASCADE', 'NONE')
(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab07$ python -m sqlite3 experiments.db "SELECT version_num FROM
 alembic_version"
('0002',)
```

Рис. 7.8. SQL-запити до бази після виконання програми

4.13. Integration tests

Тести використовують окрему SQLite-базу у `tmp_path` для кожного тесту (fixture `session_factory` створює engine, таблиці й звільняє з'єднання), тому вони не торкаються робочої `experiments.db` і не залежать один від одного. Перевіряються CRUD, зв'язок із дослідником, FOREIGN KEY і UNIQUE, каскадне видалення, агрегати SQL, фільтр за діапазоном з межами, сортування за датою, пагінація, rollback при некоректному результаті та при порушенні CHECK, DB-API, upgrade/downgrade міграцій і підкоманда `db`. Запуск з `-W error::ResourceWarning` виявив незакриті з'єднання: `with sqlite3.connect()` лише комітить, але не закриває з'єднання, тому в коді застосовано `contextlib.closing`, а engine звільняється `dispose()`.

Лістинг 7.10. Fixtures тестової бази

```
@pytest.fixture
def db_url(tmp_path: Path) -> str:
    """Separate SQLite file for every test - never the development database."""
    return f"sqlite:///{tmp_path / 'test_experiments.db'}"


@pytest.fixture
def session_factory(db_url: str) -> Iterator[sessionmaker[Session]]:
    from experiment_registry.persistence.database import create_db_engine,
create_session_factory, init_db

    engine = create_db_engine(db_url)
    init_db(engine)
    yield create_session_factory(engine)
    engine.dispose()


@pytest.fixture
def db_session(session_factory: sessionmaker[Session]) -> Iterator[Session]:
    session = session_factory()
    yield session
    session.rollback()
    session.close()
```

Лістинг 7.11. Приклади integration tests

```
@pytest.fixture
def registry(session_factory: sessionmaker[Session]) -> ExperimentRegistry:
    registry = ExperimentRegistry(session_factory)
    registry.add_series(DATA)
    return registry


def test_update_changes_result_and_notes(registry: ExperimentRegistry, db_session: Session) ->
None:
    registry.update_result(2, 102.0)
    ExperimentRepository(db_session).update(2, notes="calibrated")
    db_session.commit()

    assert db_session.get(ExperimentORM, 2).result == 102.0  # type: ignore[union-attr]
    assert db_session.get(ExperimentORM, 2).notes == "calibrated"  # type: ignore[union-attr]


def test_foreign_key_is_enforced(db_session: Session) -> None:
    db_session.add(ExperimentORM(name="Orphan", researcher_id=999, parameter="temperature",
                                 result=1.0, date=date(2026, 1, 1)))

    with pytest.raises(IntegrityError, match="FOREIGN KEY"):
        db_session.flush()


def test_invalid_record_rolls_back_whole_series(registry: ExperimentRegistry) -> None:
```

```
    series = [NewExperiment("Fuel cell", "Iryna Melnyk", "voltage", 0.71, date(2026, 3, 13)),
              NewExperiment("Broken sensor", "Maria Shevchuk", "temperature", -5.0, date(2026,
3, 18))]

    with pytest.raises(ExperimentValidationError, match="outside"):
        registry.add_series(series)

    assert registry.statistics().count == len(DATA)
    assert registry.find_by_researcher("Iryna Melnyk") == []  # researcher insert rolled back
too
```

Лістинг 7.12. Тести DB-API і міграцій

```
def test_sql_injection_is_treated_as_plain_value(connection: sqlite3.Connection) -> None:
    assert dbapi.search_by_researcher(connection, "x' OR '1'='1") == []


def test_migrations_upgrade_and_downgrade(tmp_path: Path, monkeypatch: pytest.MonkeyPatch) ->
None:
    url = f"sqlite:///{tmp_path / 'migrated.db'}"
    monkeypatch.delenv("DATABASE_URL", raising=False)
    config = alembic_config(url)

    command.upgrade(config, "head")
    columns = table_columns(url, "experiments")
    assert columns is not None
    assert {"id", "name", "researcher_id", "parameter", "result", "date", "notes"} <= columns

    command.downgrade(config, "0001")
    columns = table_columns(url, "experiments")
    assert columns is not None and "notes" not in columns

    command.downgrade(config, "base")
    assert table_columns(url, "experiments") is None
```

```
q3efirok@fedora: ~/Documents/Унік/ПП

(.venv) q3efirok@fedora:~/Documents/Унік/ПП/lab07$ pytest -v tests/integration/test_persistence.py
tests/integration/test_dbapi_and_migrations.py
====================================== test session starts ======================================
platform linux -- Python 3.14.8, pytest-9.2.0, pluggy-1.7.0 -- /tmp/claude-1000/-home-q3efirok-
Documents--------/2c66a4d4-653f-44c1-93e4-06cc10cadde1/scratchpad/venv/bin/python
rootdir: /home/q3efirok/Documents/Унік/ПП/lab07
configfile: pyproject.toml
plugins: asyncio-1.4.1.post1, cov-7.1.0, anyio-4.16.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=function,
asyncio_default_test_loop_scope=function
collecting ... collected 27 items

tests/integration/test_persistence.py::test_create_assigns_primary_key_and_researcher PASSED [  3%]
tests/integration/test_persistence.py::test_read_returns_saved_values PASSED                [  7%]
tests/integration/test_persistence.py::test_update_changes_result_and_notes PASSED          [ 11%]
tests/integration/test_persistence.py::test_update_rejects_unknown_field PASSED             [ 14%]
tests/integration/test_persistence.py::test_delete_removes_row PASSED                       [ 18%]
tests/integration/test_persistence.py::test_researcher_relationship_and_reuse PASSED        [ 22%]
tests/integration/test_persistence.py::test_foreign_key_is_enforced PASSED                  [ 25%]
tests/integration/test_persistence.py::test_deleting_researcher_cascades_to_experiments PASSED [ 29%]
tests/integration/test_persistence.py::test_unique_researcher_name_is_enforced PASSED       [ 33%]
tests/integration/test_persistence.py::test_find_by_researcher PASSED                       [ 37%]
tests/integration/test_persistence.py::test_aggregates_are_calculated_by_sql PASSED         [ 40%]
tests/integration/test_persistence.py::test_aggregates_of_empty_table PASSED                [ 44%]
tests/integration/test_persistence.py::test_range_filter_includes_boundaries[290-302-expected0] PASSED [ 48%]
tests/integration/test_persistence.py::test_range_filter_includes_boundaries[77.2-77.2-expected1] PASSED [ 51%]
tests/integration/test_persistence.py::test_range_filter_includes_boundaries[400-500-expected2] PASSED [ 55%]
tests/integration/test_persistence.py::test_experiments_are_ordered_by_date PASSED          [ 59%]
tests/integration/test_persistence.py::test_statistics_by_researcher_uses_join_and_group_by PASSED [ 62%]
tests/integration/test_persistence.py::test_pagination PASSED                               [ 66%]
tests/integration/test_persistence.py::test_invalid_record_rolls_back_whole_series PASSED   [ 70%]
tests/integration/test_persistence.py::test_database_constraint_violation_rolls_back PASSED [ 74%]
tests/integration/test_dbapi_and_migrations.py::test_parameterized_search_by_researcher_orders_by_date PASSED [ 77%]
tests/integration/test_dbapi_and_migrations.py::test_sql_injection_is_treated_as_plain_value PASSED [ 81%]
tests/integration/test_dbapi_and_migrations.py::test_range_query_and_aggregates PASSED      [ 85%]
tests/integration/test_dbapi_and_migrations.py::test_raw_schema_enforces_foreign_key PASSED [ 88%]
tests/integration/test_dbapi_and_migrations.py::test_migrations_upgrade_and_downgrade PASSED [ 92%]
tests/integration/test_dbapi_and_migrations.py::test_db_command_runs_demo_on_migrated_database PASSED [ 96%]
tests/integration/test_dbapi_and_migrations.py::test_db_command_without_schema_returns_two PASSED [100%]

====================================== 27 passed in 0.36s =======================================
```

Рис. 7.9. Результат pytest -v для persistence layer

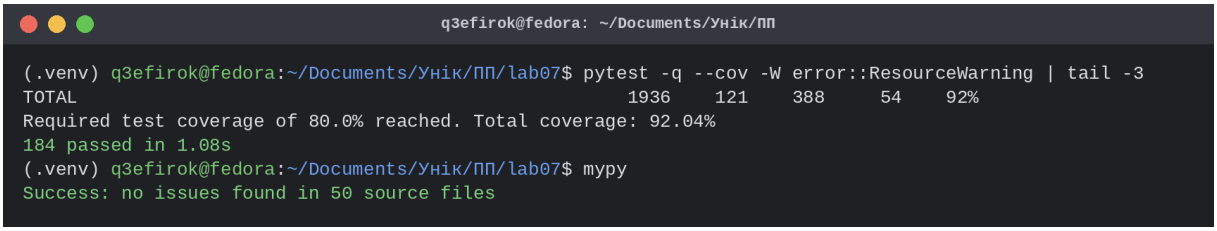


Рис. 7.10. Увесь test suite з coverage і перевірка mypy

4.14. Завдання підвищеної складності

- JOIN і GROUP BY (`statistics_by_researcher`), COUNT/AVG/MIN/MAX;
- сортування за датою і пагінація (`limit`, `offset`);
- UNIQUE і CHECK constraints, обробка `IntegrityError`, каскадне видалення;
- SQLite foreign key enforcement і integration tests для constraints;
- міграції з upgrade і downgrade, fixture для тестової БД, integration test rollback;
- unit of work (`session_scope`).

5. Відповіді на контрольні запитання

1. Що таке база даних?

База даних — це структурована сукупність взаємопов'язаних даних, організована так, щоб їх можна було ефективно зберігати, шукати, оновлювати, видаляти, фільтрувати й агрегувати. Керує нею СУБД, яка забезпечує цілісність, паралельний доступ і збереження даних між запусками програми. У моєму проєкті база зберігає дослідників і результати їхніх наукових експериментів.

2. Що таке реляційна база даних?

Реляційна база даних зберігає дані у вигляді таблиць (відношень), пов'язаних між собою через ключі: первинний ключ ідентифікує рядок, а зовнішній посилається на рядок іншої таблиці. Робота з нею ведеться мовою SQL, а цілісність забезпечують обмеження PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL. Наприклад, таблиця experiments пов'язана з researchers через researcher_id.

3. Що таке таблиця?

Таблиця — основна структура реляційної БД, яка описує один тип сутностей: вона має фіксований набір іменованих стовпців із типами та обмеженнями і довільну кількість рядків-записів. У проєкті є дві таблиці: researchers (дослідники) та experiments (експерименти).

4. Що таке рядок таблиці?

Рядок (запис, row) — це один екземпляр сутності в таблиці, який містить значення для кожного стовпця і зазвичай однозначно ідентифікується первинним ключем. Наприклад, один рядок таблиці experiments описує один експеримент: його id, name, researcher_id, parameter, result, date і notes.

5. Що таке поле?

Поле (стовпець, column, атрибут) — іменована характеристика сутності з визначеним типом даних і обмеженнями, наприклад INTEGER, TEXT, NOT NULL

чи UNIQUE; на перетині рядка й стовпця знаходиться конкретне значення. У таблиці researchers полями є id, full_name, email і department.

6. Що таке SQL?

SQL (Structured Query Language) — декларативна мова для роботи з реляційними БД: програміст описує, що потрібно отримати чи змінити, а СУБД сама вирішує, як це виконати. Команди поділяють на DDL (CREATE, ALTER, DROP), DML (INSERT, UPDATE, DELETE), DQL (SELECT) і TCL (COMMIT, ROLLBACK).

7. Для чого використовується CREATE TABLE?

CREATE TABLE — DDL-команда для створення нової таблиці: вона задає назву таблиці, стовпці з типами та обмеження цілісності (PRIMARY KEY, NOT NULL, UNIQUE, FOREIGN KEY, DEFAULT). Форма CREATE TABLE IF NOT EXISTS не спричиняє помилки, якщо таблиця вже існує. У проєкті таблиці researchers і experiments створює міграція Alembic 0001.

8. Що означає CRUD?

CRUD — абревіатура чотирьох базових операцій над даними: Create (створення), Read (читання), Update (оновлення) і Delete (видалення). У SQL їм відповідають INSERT, SELECT, UPDATE і DELETE, а в REST API — методи POST, GET, PUT/PATCH і DELETE. У проєкті CRUD-операції інкапсульовано в ResearcherRepository та ExperimentRepository.

9. Яка SQL-команда відповідає Create?

Операції Create відповідає команда INSERT INTO, яка додає новий рядок у таблицю, наприклад INSERT INTO researchers (full_name, email, department) VALUES (?, ?, ?). Значення передаються через параметри, а не вставляються в текст запиту; id типу INTEGER PRIMARY KEY SQLite генерує автоматично.

10. Яка команда відповідає Read?

Операції Read відповідає команда SELECT, яка вибирає дані з однієї чи кількох таблиць і може містити WHERE, JOIN, GROUP BY, ORDER BY та LIMIT. Наприклад, SELECT * FROM experiments WHERE researcher_id = ? ORDER BY date повертає експерименти дослідника в хронологічному порядку; SELECT не змінює даних.

11. Яка команда відповідає Update?

Операції Update відповідає команда UPDATE ... SET ... WHERE ..., яка змінює значення полів в існуючих рядках, наприклад UPDATE experiments SET result = ? WHERE id = ?. Умова WHERE критично важлива: без неї буде оновлено всі рядки таблиці.

12. Яка команда відповідає Delete?

Операції Delete відповідає команда DELETE FROM ... WHERE ..., яка видаляє рядки, що задовольняють умову, наприклад DELETE FROM researchers WHERE id = ?. Без WHERE буде видалено всі рядки; у проєкті завдяки ON DELETE CASCADE разом із дослідником автоматично видаляються і всі його експерименти.

13. Що таке SQLite?

SQLite — компактна вбудована (embedded) реляційна СУБД, яка не потребує окремого серверного процесу й зберігає всю базу даних в одному файлі на диску. У Python вона доступна через стандартний модуль sqlite3 і через SQLAlchemy; у проєкті SQLite є сховищем даних про дослідників та експерименти.

14. Які переваги SQLite?

SQLite не потребує встановлення й налаштування сервера, входить у стандартну бібліотеку Python (sqlite3), зберігає базу в одному переносному файлі, підтримує більшу частину SQL та ACID-транзакції. Вона зручна для локальних застосунків, прототипів і тестів, наприклад окремий файл БД у tmp_path. Обмеження: лише один записувач одночасно, а зовнішні ключі треба вмикати через PRAGMA foreign_keys = ON.

15. Що таке Python DB-API?

Python DB-API (PEP 249) — стандартний інтерфейс доступу Python-програм до реляційних БД, який описує функцію connect(), об'єкти Connection і Cursor, методи execute(), fetchone(), fetchall(), commit(), rollback() та ієрархію винятків. Завдяки йому драйвери різних СУБД мають однаковий API; у проєкті DB-API-модуль використовує sqlite3.

16. Для чого використовується Connection?

Connection представляє з'єднання з базою даних і створюється викликом sqlite3.connect() з шляхом до файлу. Через нього створюють курсори, керують транзакціями методами commit() і rollback() та закривають з'єднання методом close(); у sqlite3 його можна використати в with, що автоматично робить commit або rollback, але не закриває з'єднання.

17. Для чого використовується Cursor?

Cursor — об'єкт, отриманий через connection.cursor(), за допомогою якого виконують SQL-запити методами execute() та executemany() і читають результати через fetchone(), fetchall() або ітерацію. Він також має атрибути rowcount (кількість змінених рядків) і lastrowid (id щойно вставленого рядка).

18. Для чого потрібен commit()?

commit() фіксує поточну транзакцію: усі зміни, зроблені INSERT, UPDATE чи DELETE, стають постійними й видимими для інших з'єднань. Без commit() у sqlite3 зміни буде втрачено при закритті з'єднання. У проєкті session_scope викликає commit, якщо блок with завершився без винятку.

19. Для чого потрібен rollback()?

rollback() скасовує всі незафіксовані зміни поточної транзакції й повертає базу до стану після останнього commit(). Його викликають в обробнику винятків, щоб у БД не залишилося частково виконаних операцій; у проєкті session_scope робить rollback при винятку, тому ExperimentRegistry.add_series не зберігає жодного експерименту серії, якщо один результат некоректний.

20. Що таке SQL injection?

SQL injection — вразливість, за якої дані користувача вставляються безпосередньо в текст SQL-запиту й змінюють його логіку. Наприклад, рядок ' OR '1'='1, підставлений у WHERE full_name = '...', робить умову завжди істинною, а шкідливий ввід може прочитати чужі дані чи видалити таблиці. Захист — параметризовані запити або ORM.

21. Що таке parameterized query?

Parameterized query — запит, у тексті якого замість значень стоять плейсхолдери, а самі значення передаються окремо, і драйвер прив'язує їх як дані, а не як частину SQL-коду. У sqlite3 це позиційні ? з кортежем або іменовані :name зі словником, наприклад cursor.execute("SELECT * FROM experiments WHERE researcher_id = ?", (researcher_id,)).

22. Чому не рекомендується формувати SQL через f-string?

У f-string значення стає частиною тексту SQL, тому лапки, коментарі -- чи крапка з комою у вводі можуть змінити сам запит і призвести до SQL injection. Крім того, доводиться вручну екранувати лапки, виникають помилки з типами й NULL, а СУБД не може повторно використати підготовлений запит. Тому значення передають лише через параметри ? чи :name.

23. Що таке Primary Key?

Primary Key (первинний ключ) — стовпець або набір стовпців, значення яких однозначно ідентифікують кожен рядок таблиці; він унікальний, не може бути NULL, і в таблиці може бути лише один такий ключ. У проєкті це id INTEGER PRIMARY KEY, який у SQLite є псевдонімом rowid і генерується автоматично.

24. Що таке Foreign Key?

Foreign Key (зовнішній ключ) — стовпець, значення якого мають посилатися на існуючий первинний або унікальний ключ іншої таблиці, що забезпечує посилальну цілісність. У проєкті experiments.researcher_id посилається на researchers.id з ON DELETE CASCADE, а в SQLite перевірку треба вмикати для кожного з'єднання командою PRAGMA foreign_keys = ON.

25. Що таке транзакція?

Транзакція — група SQL-операцій, яка виконується як єдине неподільне ціле: або всі зміни фіксуються через COMMIT, або при помилці всі скасовуються через ROLLBACK. У проєкті ExperimentRegistry.add_series додає серію експериментів в одній транзакції, тож якщо результат хоча б одного з них некоректний, у БД не потрапляє жоден.

26. Що означає Atomicity?

Atomicity (атомарність) означає, що транзакція неподільна: усі її операції виконуються повністю або не виконується жодна, і проміжний стан не зберігається. Реалізується через commit/rollback; наприклад, при помилці в ExperimentRegistry.add_series відкат скасовує і ті експерименти серії, які вже були вставлені в межах транзакції.

27. Що означає Consistency?

Consistency (узгодженість) означає, що транзакція переводить базу з одного коректного стану в інший і не порушує обмежень цілісності: PRIMARY KEY, UNIQUE, NOT NULL, FOREIGN KEY, CHECK. Якщо операція порушує правило, наприклад вставляє експеримент із неіснуючим researcher_id чи дублює full_name дослідника, СУБД її відхиляє, а транзакцію відкочують.

28. Що означає Isolation?

Isolation (ізольованість) означає, що паралельні транзакції не повинні некоректно впливати одна на одну: незафіксовані зміни однієї не видно іншим, а результат має бути таким, ніби вони виконувалися послідовно. Ступінь ізоляції задається рівнями (READ COMMITTED, SERIALIZABLE тощо); SQLite за замовчуванням забезпечує серіалізовану ізоляцію, дозволяючи лише одного записувача одночасно.

29. Що означає Durability?

Durability (довговічність) означає, що після успішного commit() зміни зберігаються постійно й не втрачаються навіть при збої програми, вимкненні живлення чи перезапуску. СУБД досягає цього записом на диск і журналюванням, у SQLite — через rollback journal або WAL.

30. Що означає ACID?

ACID — чотири властивості надійних транзакцій: Atomicity (усе або нічого), Consistency (збереження коректного стану й обмежень), Isolation (паралельні транзакції не заважають одна одній) і Durability (підтверджені зміни не втрачаються). SQLite підтримує ACID-транзакції, а в проєкті вони керуються через session_scope з commit і rollback.

31. Що таке persistence layer?

Persistence layer (шар збереження даних) — частина застосунку, яка відповідає за збереження й завантаження даних із постійного сховища та приховує від бізнес-логіки деталі SQL, з'єднань і транзакцій. У проєкті це пакет experiment_registry.persistence з ORM-моделями ResearcherORM і ExperimentORM, репозиторіями та контекстним менеджером session_scope.

32. Для чого потрібний Repository Pattern?

Repository Pattern відокремлює бізнес-логіку від способу зберігання даних: репозиторій надає колекцієподібний інтерфейс (додати, знайти за id, отримати всі, оновити, видалити) й інкапсулює всі запити до БД. Сервіс не знає, чи працює SQLAlchemy, DB-API чи інша СУБД, тому код легше тестувати й змінювати; у проєкті це ResearcherRepository та ExperimentRepository.

33. Чому Repository відокремлюють від Service Layer?

Repository відповідає лише за доступ до даних (запити, збереження, вибірки), а Service Layer — за бізнес-правила, валідацію та координацію кількох операцій у межах транзакції. Таке розділення реалізує принцип єдиної відповідальності: можна замінити сховище без зміни бізнес-логіки й тестувати сервіс із фейковим репозиторієм. Наприклад, ExperimentRegistry.add_series перевіряє серію й керує транзакцією, а запис виконують репозиторії.

34. Що таке ORM?

ORM (Object-Relational Mapping) — техніка відображення таблиць реляційної БД на класи, а рядків — на об'єкти, завдяки чому з даними працюють через атрибути й методи об'єктів замість ручного SQL. ORM сама генерує параметризовані запити, відстежує зміни об'єктів у сесії та керує зв'язками; у Python найпоширеніша ORM — SQLAlchemy.

35. Для чого використовується SQLAlchemy?

SQLAlchemy — бібліотека Python для роботи з реляційними БД, що складається з Core (Engine, пул з'єднань, SQL Expression Language) та ORM (декларативні моделі, Session, relationship). Вона дозволяє одним кодом працювати з SQLite, PostgreSQL, MySQL тощо; у проєкті використовується SQLAlchemy 2.0 ORM з моделями ResearcherORM і ExperimentORM.

36. Що таке ORM model?

ORM model — Python-клас, успадкований від декларативної бази (у SQLAlchemy 2.0 — класу на основі DeclarativeBase), який описує таблицю: __tablename__ задає її назву, а атрибути Mapped[...] = mapped_column(...) — стовпці з типами, ключами й обмеженнями. Кожен екземпляр відповідає рядку таблиці; у проєкті ResearcherORM відображає таблицю researchers, а ExperimentORM — experiments.

37. Що таке relationship()?

relationship() — функція SQLAlchemy ORM, яка оголошує зв'язок між моделями на рівні об'єктів, дозволяючи звертатися до пов'язаних об'єктів як до атрибута, тоді як сам зв'язок у БД задає ForeignKey. Параметр back_populates синхронізує обидві сторони; у проєкті ResearcherORM має колекцію експериментів, а ExperimentORM — посилання на свого дослідника.

38. Що означає one-to-many?

One-to-many (один-до-багатьох) — зв'язок, за якого одному запису першої таблиці відповідає багато записів другої, а кожен запис другої пов'язаний лише з одним записом першої. Реалізується зовнішнім ключем на стороні «багатьох»: у проєкті один дослідник має багато експериментів, а experiments.researcher_id посилається на researchers.id.

39. Що означає many-to-many?

Many-to-many (багато-до-багатьох) — зв'язок, за якого кожен запис однієї таблиці може бути пов'язаний з багатьма записами іншої і навпаки. У реляційній БД він реалізується через проміжну (асоціативну) таблицю з двома зовнішніми ключами, а в SQLAlchemy — через relationship() з параметром secondary; так було б, якби один експеримент могли виконувати кілька дослідників.

40. Що таке migration?

Migration (міграція) — версіонований скрипт, що описує зміну структури БД (створення таблиць, додавання стовпців, індексів) функціями upgrade() і downgrade(), завдяки чому схему змінюють без видалення бази й втрати даних. У проєкті міграція 0001 створює таблиці researchers і experiments, а 0002 додає стовпець notes.

41. Для чого використовується Alembic?

Alembic — інструмент міграцій для SQLAlchemy, який зберігає історію змін схеми як ланцюжок ревізій, застосовує та відкочує їх і записує поточну версію в службову таблицю alembic_version. Команда alembic revision --autogenerate створює чернетку міграції, порівнюючи ORM-моделі зі схемою БД; у проєкті Alembic керує міграціями 0001 і 0002.

42. Для чого потрібна команда alembic upgrade head?

Команда alembic upgrade head застосовує до бази по черзі всі ще не виконані міграції до найновішої ревізії (head), викликаючи їхні функції upgrade(), і оновлює номер версії в таблиці alembic_version. Для нової БД проєкту вона послідовно виконає 0001 і 0002; відкотити останню міграцію можна командою alembic downgrade -1.

43. Що таке integration test?

Integration test (інтеграційний тест) перевіряє спільну роботу кількох компонентів із реальними залежностями, наприклад сервісу, репозиторію, ORM і справжньої бази даних. У проєкті інтеграційні тести працюють з окремим файлом SQLite у tmp_path і перевіряють реальне збереження, читання, оновлення й видалення даних та відкат транзакцій.

44. Чим integration test відрізняється від unit test?

Unit test перевіряє одну ізольовану одиницю коду (функцію, клас) без зовнішніх залежностей, які підміняються моками чи фейками, тому він швидкий і точно локалізує помилку. Integration test перевіряє взаємодію компонентів із реальною інфраструктурою, наприклад репозиторію разом із SQLAlchemy та файлом SQLite, тому він повільніший, але виявляє помилки на стиках: у SQL, мапінгу, обмеженнях і транзакціях.

45. Навіщо використовувати окрему test database?

Окрема test database ізолює тести від робочих даних: тести можуть вільно створювати й видаляти записи, не пошкоджуючи реальну БД, і завжди стартують із відомого чистого стану, тому результати відтворювані й не залежать від порядку запуску. У проєкті кожен тест отримує власний файл SQLite в унікальному тимчасовому каталозі tmp_path.

46. Що перевіряє тест rollback?

Тест rollback перевіряє атомарність транзакції: якщо під час неї виникає помилка, жодна зі змін не повинна залишитися в БД. У проєкті він викликає ExperimentRegistry.add_series із серією, де один результат некоректний, і перевіряє, що виникає виняток, а в базі немає жодного експерименту цієї серії, тобто вже вставлені записи відкочено.

47. Для чого використовуються fixtures?

Fixtures у pytest — функції з декоратором @pytest.fixture, які готують і передають тестам ресурси (БД, сесію, репозиторій, тестові дані), а код після yield виконує очищення. Тест отримує фікстуру через однойменний параметр, а scope визначає, як

часто її створювати; у проєкті на основі вбудованої фікстури tmp_path готується окремий файл SQLite для інтеграційних тестів.

48. Що робить SQL JOIN?

JOIN об'єднує рядки двох або більше таблиць за умовою зв'язку, зазвичай рівністю зовнішнього та первинного ключів. INNER JOIN повертає лише рядки з відповідністю, а LEFT JOIN — усі рядки лівої таблиці, заповнюючи відсутні значення NULL; наприклад, SELECT e.name, r.full_name FROM experiments e JOIN researchers r ON e.researcher_id = r.id показує експерименти з іменами дослідників.

49. Що робить GROUP BY?

GROUP BY групує рядки з однаковими значеннями вказаних стовпців, щоб агрегатні функції обчислювалися окремо для кожної групи; умови на результати агрегації задаються через HAVING. Наприклад, SELECT researcher_id, COUNT(*), AVG(result) FROM experiments GROUP BY researcher_id повертає кількість і середній результат експериментів кожного дослідника.

50. Для чого використовуються COUNT, SUM, AVG, MIN, MAX?

Це агрегатні функції, які повертають одне значення для всього набору рядків або для кожної групи GROUP BY: COUNT — кількість рядків чи не-NULL значень, SUM — суму, AVG — середнє, MIN і MAX — мінімум і максимум. Усі, крім COUNT(*), ігнорують NULL; у DB-API-модулі проєкту AVG, MIN, MAX і COUNT з GROUP BY обчислюють статистику результатів експериментів.

6. Висновок

У лабораторній роботі проєкт experiment-registry отримав persistence layer на SQLite: дві пов’язані таблиці з первинними та зовнішнім ключем, ORM-моделі SQLAlchemy, репозиторії для дослідників і експериментів, сервіс із транзакційними операціями та модуль прямих параметризованих SQL-запитів через DB-API. Збережено функції лабораторної №1 (додавання, пошук за дослідником, середнє/мінімум/максимум, фільтрація за діапазоном) і додано CRUD, сортування за датою, агрегати та статистику з JOIN і GROUP BY.

Схемою керують міграції Alembic (0001 — таблиці, 0002 — колонка notes), які застосовуються і відкочуються командами upgrade/downgrade. Пакетне додавання серії експериментів виконується атомарно: некоректний результат скасовує всю серію. 27 integration tests на окремій тестовій базі перевіряють CRUD, зв’язки, обмеження, агрегати, сортування, rollback і міграції; увесь набір із 184 тестів проходить, а тести допомогли знайти дві реальні проблеми (вимкнення логерів Alembic і незакриті з’єднання sqlite3).